

▼ STEP 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
```

▼ STEP 2: Load Dataset

```
df = pd.read_excel("/content/Dataset_Final_Project from PPDA.xlsx")
```

```
# View first rows
print(df.head())
```

[Show hidden output](#)

```
# Check structure
print(df.info())
```

[Show hidden output](#)

▼ STEP 3: Data Description

```
# Summary statistics
print(df.describe())
```

[Show hidden output](#)

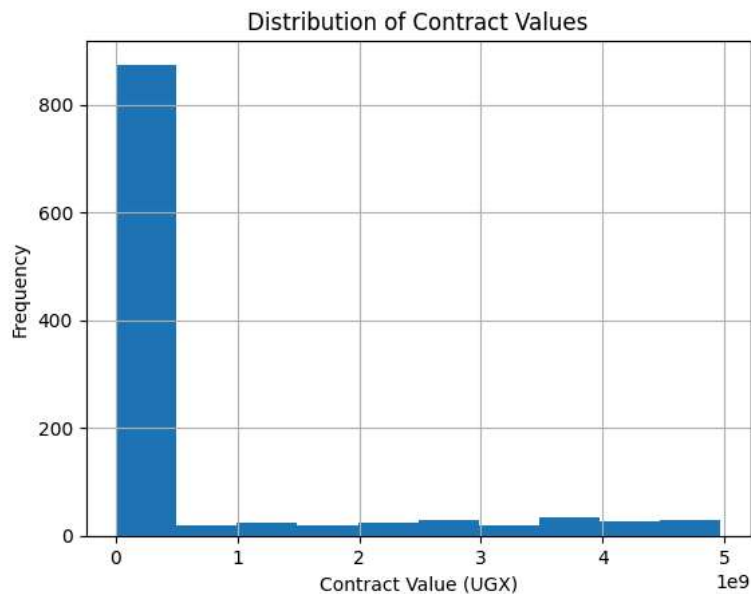
```
# Check unique values
print(df['Status'].value_counts())
print(df['Procurement_Method'].value_counts())
```

[Show hidden output](#)

▼ STEP 4: Data Visualization

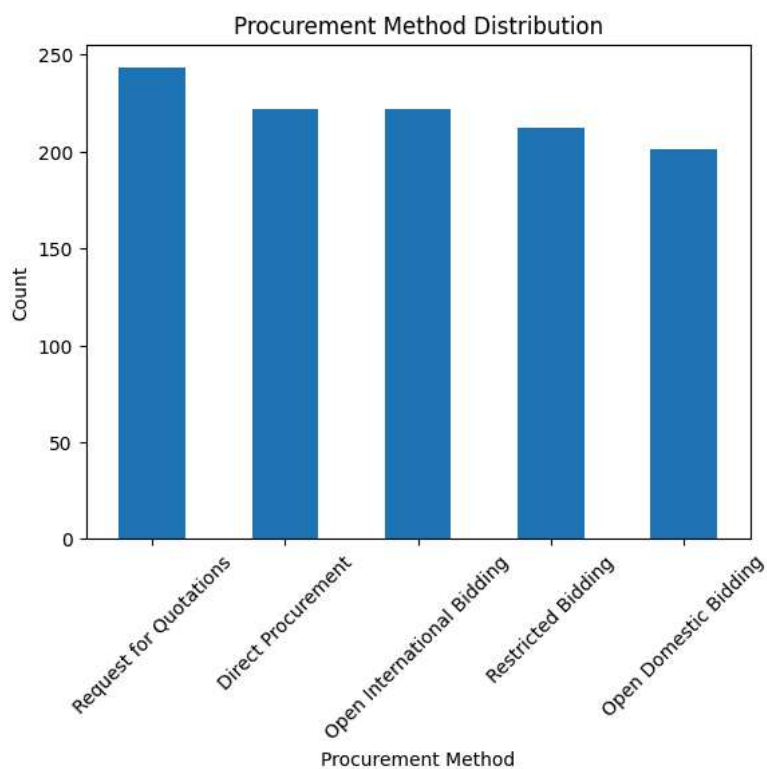
4.1 Univariate – Contract Value Distribution

```
plt.figure()
df['Contract_Value_UGX'].hist()
plt.xlabel("Contract Value (UGX)")
plt.ylabel("Frequency")
plt.title("Distribution of Contract Values")
plt.show()
```



4.2 Univariate – Procurement Method

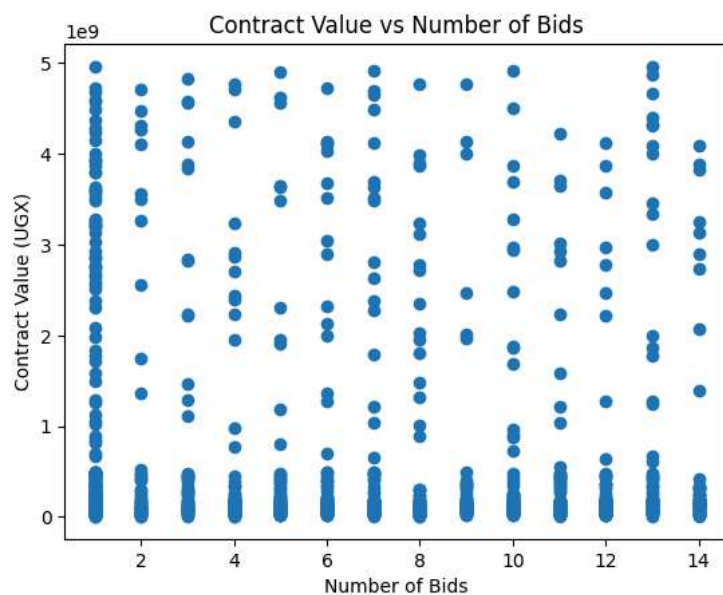
```
plt.figure()
df['Procurement_Method'].value_counts().plot(kind='bar')
plt.xlabel("Procurement Method")
plt.ylabel("Count")
plt.title("Procurement Method Distribution")
plt.xticks(rotation=45)
plt.show()
```



4.3 Multivariate – Value vs Bids

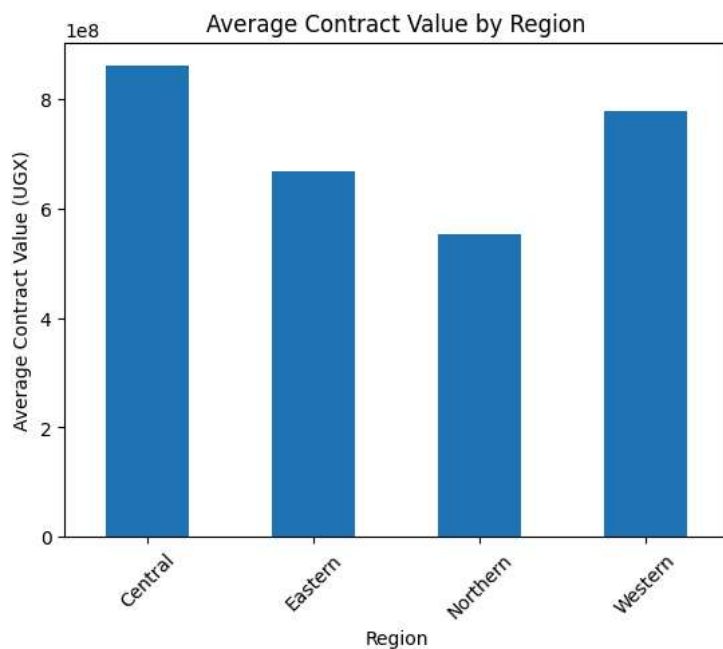
```
plt.figure()
plt.scatter(df['Number_of_Bids'], df['Contract_Value_UGX'])
plt.xlabel("Number of Bids")
plt.ylabel("Contract Value (UGX)")
```

```
plt.title("Contract Value vs Number of Bids")
plt.show()
```



4.4 Multivariate – Region vs Value

```
plt.figure()
df.groupby('Region')['Contract_Value_UGX'].mean().plot(kind='bar')
plt.xlabel("Region")
plt.ylabel("Average Contract Value (UGX)")
plt.title("Average Contract Value by Region")
plt.xticks(rotation=45)
plt.show()
```



▼ STEP 5: Data Cleaning & Preparation

```
df_ml = df.copy()

# Encode categorical variables
label_encoders = {}

categorical_cols = [
```

```

        'Procuring_Entity',
        'Category',
        'Item_Description',
        'Procurement_Method',
        'Region'
    ]

    for col in categorical_cols:
        le = LabelEncoder()
        df_ml[col] = le.fit_transform(df_ml[col])
        label_encoders[col] = le

    # Encode target variable
    le_status = LabelEncoder()
    df_ml['Status'] = le_status.fit_transform(df_ml['Status'])

```

▼ STEP 6: Feature Selection

```

X = df_ml.drop(['Contract_ID', 'Status'], axis=1)
y = df_ml['Status']

```

▼ STEP 7: Feature Scaling

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

▼ STEP 8: Train-Test Split

```

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

```

▼ STEP 9: Classical ML Model (Random Forest)

```

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

rf_preds = rf.predict(X_test)

print("Random Forest Results:")
print(classification_report(y_test, rf_preds))

```

```

Random Forest Results:
              precision    recall  f1-score   support

     0       0.70      0.98      0.82      154
     1       0.40      0.04      0.07       54
     2       0.00      0.00      0.00       12

 accuracy          0.70          220
 macro avg       0.37      0.34      0.30      220
 weighted avg    0.59      0.70      0.59      220

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 0: no predicted samples
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 1: no predicted samples
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 2: no predicted samples
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

▼ STEP 10: Neural Network Model

```
mlp = MLPClassifier(max_iter=500)

mlp.fit(X_train, y_train)

mlp_preds = mlp.predict(X_test)

print("Neural Network Results:")
print(classification_report(y_test, mlp_preds))
```

```
Neural Network Results:
              precision    recall  f1-score   support

     0       0.70      0.97      0.81       154
     1       0.20      0.02      0.03        54
     2       0.00      0.00      0.00         12

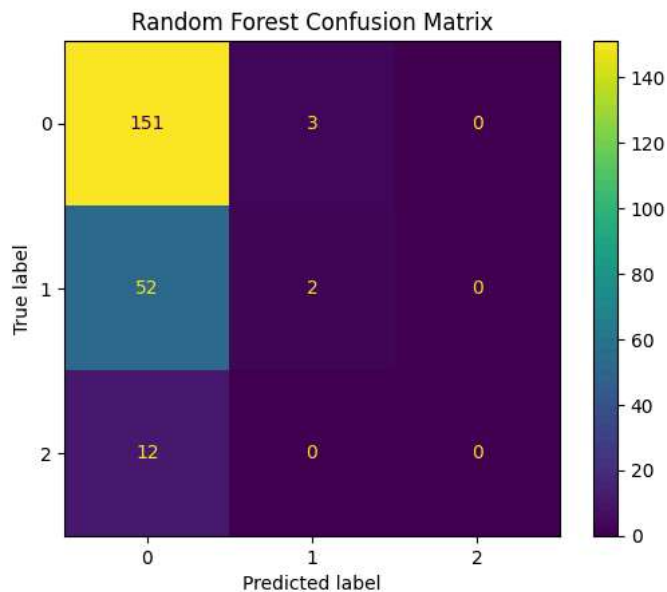
 accuracy          0.30      0.33      0.28       220
 macro avg          0.30      0.33      0.28       220
 weighted avg          0.54      0.69      0.58       220
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Opt
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

STEP 11: Model Evaluation [Confusion Matrix]

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test)
plt.title("Random Forest Confusion Matrix")
plt.show()
```



WORKING DEMO:

```
# --- STEP: WORKING DEMO ---
import ipywidgets as widgets
from IPython.display import display, HTML

# 1. Setup UI Styling
style = {'description_width': 'initial'}
header = HTML("<h2>🚧 Procurement Status Prediction Demo</h2>")
display(header)
```

```

# 2. Dynamically Create Input Widgets for Features
input_widgets = {}

# We iterate through the columns used in training (X.columns)
for col in X.columns:
    if col in categorical_cols:
        # Create a dropdown for categorical features using the encoder classes
        input_widgets[col] = widgets.Dropdown(
            options=label_encoders[col].classes_,
            description=f'{col.replace("_", " ")}:',
            style=style,
            layout={'width': '400px'})
    else:
        # Create a numeric input for numerical features
        input_widgets[col] = widgets.FloatText(
            value=float(df[col].median()), # Default to median value
            description=f'{col.replace("_", " ")}:',
            style=style,
            layout={'width': '400px'})

# 3. Create Prediction Button and Output Area
predict_btn = widgets.Button(
    description="Predict Status",
    button_style='primary', # 'success', 'info', 'warning', 'danger' or ''
    tooltip='Click to predict the procurement status',
    icon='check'
)
output = widgets.Output()

# 4. Define the Prediction Logic
def perform_prediction(b):
    with output:
        output.clear_output()
        try:
            # Gather inputs into a dictionary
            data_dict = {}
            for col, widget in input_widgets.items():
                val = widget.value
                if col in categorical_cols:
                    # Transform the string input to the encoded integer
                    val = label_encoders[col].transform([val])[0]
                data_dict[col] = val

            # Convert to DataFrame to ensure correct column order
            input_df = pd.DataFrame([data_dict])[X.columns]

            # Apply the same scaling used during training
            input_scaled = scaler.transform(input_df)

            # Get prediction from the Random Forest model
            # (You can change 'rf' to 'mlp' if you prefer the Neural Network)
            pred_code = rf.predict(input_scaled)[0]
            prediction = le_status.inverse_transform([pred_code])[0]

            # Get probability scores
            probs = rf.predict_proba(input_scaled)[0]
            max_prob = max(probs) * 100

            # Display Result
            print("-" * 30)
            display(HTML(f"<h3>Result: <span style='color: #2e7d32;'>{prediction}</span></h3>"))
            print(f"Confidence: {max_prob:.2f}%")
            print("-" * 30)

        except Exception as e:
            display(HTML(f"<b style='color: red;'>Error:</b> {str(e)}"))

# Link button to function
predict_btn.on_click(perform_prediction)

# 5. Display the UI
ui_layout = widgets.VBox(list(input_widgets.values()) + [widgets.Label(""), predict_btn, output])
display(ui_layout)

```

```
import joblib
# Save the model and encoders
joblib.dump({'model': rf, 'scaler': scaler, 'encoders': label_encoders, 'target_encoder': le_status}, 'procurement_model.pkl')
```



Procurement Status Prediction Demo

Procuring Entity:

Category:

Item Description:

Procurement Method:

Contract Value UGX:

Number of Bids:

Region:

Predict Status

['procurement_model.pkl']